

# C语言指针与链表专项训练 - Day 3

参考：《C程序设计》（谭浩强）

- 题型：单项选择题
- 题量：50题
- 重点：指针、数组与指针、字符指针、动态内存、单链表
- 综合内容：函数、循环、输入输出
- 本卷不涉及时间复杂度

## 答题说明

每题只有一个正确答案。建议独立完成，不运行代码。把不确定的题目在答案后加？，例如：7.B?。即使答案正确，批改时也会重点讲解。

## 一、指针、数组与字符串

### 1. 已知：

```
int a = 1, b = 2;  
const int *p = &a;
```

下列语句合法的是：

- A. `p = &b;`
- B. `*p = 8;`
- C. `(*p)++;`
- D. `p[0] = 5;`

### 2. 已知：

```
int a = 1, b = 2;  
int *const p = &a;
```

下列语句合法的是：

- A. `p = &b;`
- B. `*p = 8;`
- C. `p++;`
- D. `p = NULL;`

**3. 对声明 `const int *const p = &a;` 的正确理解是：**

- A. 可以修改 `p`，也可以通过 `p` 修改 `a`
- B. 可以修改 `p`，但不能通过 `p` 修改 `a`
- C. 不能修改 `p`，也不能通过 `p` 修改 `a`
- D. 不能修改 `p`，但可以通过 `p` 修改 `a`

**4. 执行下列代码，表达式的值是：**

```
int a[6];  
sizeof(a) / sizeof(a[0])
```

- A. 1
- B. 4
- C. 6
- D. 指针所占字节数

**5. 对下面的函数形参，函数体中的 `sizeof(a)` 等于：**

```
void fun(int a[])  
{  
    /* ... */  
}
```

- A. 实参数组占用的全部字节数
- B. `sizeof(int)`
- C. `sizeof(int *)`
- D. 0

## 6. 执行下列定义后， sizeof(s) 的值是：

```
char s[] = "abc";
```

- A. 3
- B. 4
- C. sizeof(char \*)
- D. 不确定

## 7. 对下列代码的描述正确的是：

```
char *s = "abc";  
s[0] = 'A';
```

- A. 一定得到字符串 "Abc"
- B. 一定编译失败
- C. 试图修改字符串字面量，行为未定义
- D. s 会自动变成字符数组

## 8. 下列声明中，表示“指向含4个 int 元素的一维数组的指针”的是：

- A. int \*p[4];
- B. int (\*p)[4];
- C. int \*\*p[4];
- D. int p\*[4];

## 9. 已知：

```
int a[3][4];  
int (*p)[4] = a;
```

表达式  $*(*(p + 2) + 1)$  等价于：

- A. a[1][2]
- B. a[2][0]
- C. a[2][1]
- D. a[1][3]

## 10. 声明 `int *p[4];` 表示：

- A. `p` 是一个含4个元素的数组，每个元素都是 `int *`
- B. `p` 是指向含4个 `int` 元素数组的指针
- C. `p` 是二级指针
- D. `p` 是指向第4个整数的指针

## 11. 执行下列代码后，`a` 的值是：

```
int a = 3;
int *p = &a;
int **pp = &p;
**pp = 9;
```

- A. 3
- B. 9
- C. `p` 的地址
- D. 不确定

## 12. 下列程序片段的输出是：

```
int a[] = {10, 20, 30};
int *p = a;
printf("%d ", *p++);
printf("%d", *p);
```

- A. 10 10
- B. 10 20
- C. 20 20
- D. 20 30

## 13. 执行下列代码后，`x` 和 `a` 的值分别是：

```
int a = 5;
int *p = &a;
int x = (*p)++;
```

- A. `x=6`, `a=6`
- B. `x=5`, `a=6`

- C. x=5, a=5
- D. x=6, a=5

**14. 已定义 `double x;`，从键盘输入一个实数并存入 `x` 的正确语句是：**

- A. `scanf("%f", &x);`
- B. `scanf("%lf", &x);`
- C. `scanf("%d", &x);`
- D. `printf("%lf", &x);`

## 二、动态内存

**15. 动态申请 `n` 个 `int` 元素空间的推荐写法是：**

- A. `int *p = malloc(n);`
- B. `int *p = malloc(n * sizeof *p);`
- C. `int p = malloc(n * sizeof(int));`
- D. `int *p = malloc(sizeof(p));`

**16. 关于 `calloc(n, sizeof(int))`，正确的是：**

- A. 只申请一个 `int` 大小的空间
- B. 申请 `n` 个 `int` 大小的连续空间，并将所分配的字节置零
- C. 申请失败时返回 `-1`
- D. 使用后不需要调用 `free`

**17. 使用 `realloc` 扩大内存时，更安全的写法是：**

- A.

```
p = realloc(p, new_size);
```

- B.

```
int *temp = realloc(p, new_size);  
if (temp != NULL)  
    p = temp;
```

- C.

```
free(p);  
realloc(p, new_size);
```

- D.

```
p++;  
realloc(p, new_size);
```

**18. 执行 `free(p);` 后，为降低误用悬空指针的风险，通常应继续执行：**

- A. `p++;`
- B. `*p = 0;`
- C. `p = NULL;`
- D. `free(p);`

**19. 为一个最多保存19个普通字符的字符串动态申请空间，正确的是：**

- A. `char *s = malloc(20 * sizeof *s);`
- B. `char *s = malloc(19);`
- C. `char s = malloc(20);`
- D. `char *s = malloc(sizeof(s));`

**20. 执行下列代码后， `q - p` 的值是：**

```
int a[5] = {1, 2, 3, 4, 5};  
int *p = a;  
int *q = p + 2;
```

- A. 0
- B. 1
- C. 2
- D. `2 * sizeof(int)`

## 三、单链表强化训练

以下各题若无特别说明，结点类型为：

```
typedef struct Node {  
    int data;  
    struct Node *next;  
} Node;
```

### 21. 单链表结点中的成员 next 用于：

- A. 保存当前结点的数据
- B. 保存下一个结点的地址
- C. 保存链表的结点总数
- D. 保存头指针变量的地址

### 22. 对不带头结点的单链表，空表条件是：

- A. head == NULL
- B. head->next == NULL
- C. head->data == 0
- D. head->next == head

### 23. 对带头结点的普通单链表，已保证 head 指向有效头结点，则空表条件是：

- A. head == NULL
- B. head->next == NULL
- C. head->data == 0
- D. head->next == head

### 24. 依次输入1、2、3，每次都使用头插法插入不带头结点的链表，最终顺序是：

- A. 1 2 3
- B. 3 2 1
- C. 1 3 2
- D. 2 3 1

**25. 依次输入1、2、3，使用尾插法建立链表，最终顺序是：**

- A. 1 2 3
- B. 3 2 1
- C. 2 1 3
- D. 3 1 2

**26. 遍历不带头结点的普通单链表，并访问包括尾结点在内的所有结点，循环条件应为：**

- A. while (p->next != NULL)
- B. while (p != NULL)
- C. while (\*p != NULL)
- D. while (p == NULL)

**27. 已知 p 指向链表中的一个结点，把 newnode 插入到 p 后面的正确顺序是：**

- A.

```
p->next = newnode;  
newnode->next = p->next;
```

- B.

```
newnode->next = p->next;  
p->next = newnode;
```

- C.

```
newnode = p->next;  
p->next = newnode;
```

- D.

```
p = newnode;  
newnode->next = NULL;
```

**28. 已知 p->next 是待删除结点，正确的删除代码是：**

- A.

```
free(p->next);  
p->next = p->next->next;
```

- B.

```
Node *q = p->next;  
p->next = q->next;  
free(q);
```

- C.

```
p->next = NULL;  
free(p);
```

- D.

```
free(p);  
p = p->next;
```

**29. 函数参数为 Node \*\*head ， 且 \*head != NULL ， 删除不带头结点链表首结点的正确代码是：**

- A.

```
Node *q = *head;  
*head = q->next;  
free(q);
```

- B.

```
free(*head);  
*head = (*head)->next;
```

- C. \*head = NULL;
- D. free(head);

**30. 删除不带头结点链表首结点的函数使用 Node \*\*head ， 主要是为了：**

- A. 修改调用函数中的头指针
- B. 让每个结点占用更少内存

- C. 避免调用 free
- D. 把链表变成数组

### 31. 下列代码执行结束后， count 表示：

```
int count = 0;
Node *p = head;
while (p != NULL) {
    count++;
    p = p->next;
}
```

- A. 数据为0的结点数
- B. 链表中的结点总数
- C. 尾结点的数据
- D. 头指针的地址

### 32. 在链表中查找第一个数据域等于 x 的结点，正确的核心循环是：

- A.

```
while (p != NULL && p->data != x)
    p = p->next;
```

- B.

```
while (p == NULL && p->data == x)
    p++;
```

- C.

```
while (p->next == p)
    free(p);
```

- D.

```
while (p != NULL)
    head = NULL;
```

### 33. 使用三个指针原地逆置不带头结点的单链表，循环开始前通常应设置：

- A. `previous = NULL; p = head;`
- B. `previous = head; p = NULL;`
- C. `previous = head->next; p = head;`
- D. `previous = p = NULL;`

### 34. 原地逆置单链表时，正确的核心操作顺序是：

- A.

```
next = p->next;  
p->next = previous;  
previous = p;  
p = next;
```

- B.

```
p = p->next;  
p->next = previous;  
previous = p;
```

- C.

```
free(p);  
p = next;
```

- D.

```
previous->next = p;  
p = p->next;
```

### 35. 一个非空单链表完成逆置后，原来的首结点将成为：

- A. 新链表的首结点
- B. 新链表的尾结点，且其 `next` 应为 `NULL`
- C. 头结点之前的结点
- D. 被自动释放的结点

### 36. 释放不带头结点的整条单链表时，正确的循环写法是：

- A.

```
while (p != NULL) {  
    Node *next = p->next;  
    free(p);  
    p = next;  
}
```

- B.

```
while (p != NULL) {  
    free(p);  
    p = p->next;  
}
```

- C. head = NULL;
- D. free(head->next);

### 37. 对带头结点的单链表，只遍历数据结点而不访问头结点，通常应令：

- A. p = head;
- B. p = head->next;
- C. p = NULL;
- D. p = &head;

### 38. 已知 tail 指向尾结点，把 newnode 接到链表末尾的正确代码是：

- A.

```
tail = newnode;  
tail->next = newnode;
```

- B.

```
newnode->next = NULL;  
tail->next = newnode;  
tail = newnode;
```

- C.

```
newnode->next = tail;  
tail = NULL;
```

- D. free(tail);

**39. 一个带头结点的链表恰好只有一个数据结点，删除该数据结点的正确操作是：**

- A.

```
Node *q = head->next;  
head->next = NULL;  
free(q);
```

- B. free(head);
- C. head = NULL;
- D.

```
free(head->next);  
free(head);
```

**40. 若函数需要在不带头结点的链表最前面插入结点，并直接修改调用函数中的 head，合适的函数声明是：**

- A. void insert\_head(Node head, int x);
- B. void insert\_head(Node \*head, int x);
- C. void insert\_head(Node \*\*head, int x);
- D. Node insert\_head(int head, int x);

## 四、函数、循环与输入输出

41. 执行下列代码后， x 的值是：

```
void add3(int *p)
{
    *p += 3;
}
```

```
int x = 5;
add3(&x);
```

- A. 3
- B. 5
- C. 8
- D. 不确定

42. 执行下列代码后， a 的值是：

```
void fun(int x)
{
    x = 20;
}
```

```
int a = 10;
fun(a);
```

- A. 10
- B. 20
- C. 0
- D. 不确定

### 43. 下列代码的输出是：

```
int fun(void)
{
    static int x = 0;
    return ++x;
}

int a = fun();
int b = fun();
printf("%d %d", a, b);
```

- A. 1 1
- B. 1 2
- C. 2 2
- D. 0 1

### 44. 执行下列代码后， count 的值是：

```
int i = 5, count = 0;
do {
    count++;
    i++;
} while (i < 3);
```

- A. 1
- B. 0
- C. 3
- D. 无限循环

### 45. 执行下列代码后， sum 的值是：

```
int i, sum = 0;
for (i = 1; i <= 5; i++) {
    if (i % 2 == 0)
        continue;
    sum += i;
}
```

- A. 6
- B. 8
- C. 9
- D. 15

**46. 已定义 `int n;`，从键盘输入一个整数并存入 `n` 的正确语句是：**

- A. `scanf("%d", n);`
- B. `scanf("%d", &n);`
- C. `scanf("%f", &n);`
- D. `printf("%d", &n);`

**47. 已定义 `char s[20];`，安全读入一个不含空白字符、长度不超过19的字符串，正确的是：**

- A. `scanf("%s", &s);`
- B. `scanf("%19s", s);`
- C. `scanf("%20c", s);`
- D. `printf("%19s", s);`

**48. 从键盘读取一个字符并立即输出该字符，正确的是：**

- A.

```
char ch;  
printf("%c", ch);
```

- B.

```
int ch = getchar();  
putchar(ch);
```

- C.

```
int ch;  
scanf("%s", &ch);
```

- D.

```
getchar(ch);  
putchar();
```

**49. 已定义二维数组 `int a[3][4];`，能够正确接收该数组的函数形参声明是：**

- A. `void fun(int **p);`
- B. `void fun(int (*p)[4]);`
- C. `void fun(int p[][]);`
- D. `void fun(int p[4][]);`

**50. 下列函数的作用是：**

```
void fun(int *a, int n)
{
    int i;
    for (i = 0; i < n; i++)
        if (a[i] < 0)
            a[i] = 0;
}
```

- A. 把数组中的正数改为0
- B. 把数组中的负数改为0
- C. 计算数组元素之和
- D. 逆置数组

## 答题卡

|        |        |        |        |        |
|--------|--------|--------|--------|--------|
| 1. __  | 2. __  | 3. __  | 4. __  | 5. __  |
| 6. __  | 7. __  | 8. __  | 9. __  | 10. __ |
| 11. __ | 12. __ | 13. __ | 14. __ | 15. __ |
| 16. __ | 17. __ | 18. __ | 19. __ | 20. __ |
| 21. __ | 22. __ | 23. __ | 24. __ | 25. __ |
| 26. __ | 27. __ | 28. __ | 29. __ | 30. __ |
| 31. __ | 32. __ | 33. __ | 34. __ | 35. __ |
| 36. __ | 37. __ | 38. __ | 39. __ | 40. __ |
| 41. __ | 42. __ | 43. __ | 44. __ | 45. __ |
| 46. __ | 47. __ | 48. __ | 49. __ | 50. __ |

完成后把填写好的答案或清晰照片发来批改。